SAVITRIBAI PHULE PUNE UNIVERSITY

Mobile Computing

COMPUTER SCIENCE
Fourth Year (2019 Pattern)

Unit-1 (NOTES)

PUNE UNIVERSITY

Unit I: Mathematical Foundation for Blockchain

1. Symmetric Key Cryptography

Definition: Symmetric key cryptography, also known as private-key or shared-secret cryptography, is an encryption system where the sender and receiver of a message share a single, common key. This single key is used for both encrypting the plaintext and decrypting the ciphertext. The security of the system relies on the secrecy of this key; if an unauthorized party gets hold of the key, they can easily decrypt all communications.

Detailed Explanation:

- **The single key is the most critical element of the system.** The security of the entire communication depends on keeping this key a secret between the communicating parties.
- **The key distribution problem is a significant challenge.** Before secure communication can begin, a secure method must be established to exchange the secret key. If the key is intercepted during distribution, all future encrypted messages can be read by the interceptor.
- **Symmetric algorithms are generally faster and more efficient.** Because they use a single key and simpler mathematical operations, they are computationally less intensive than their asymmetric counterparts, making them suitable for encrypting large volumes of data.
- **Common algorithms include AES (Advanced Encryption Standard) and DES (Data Encryption Standard).** AES is currently the most widely used and recommended symmetric algorithm for securing data.

Mathematical / Technical Aspect:

- **Encryption Process:** C=E_k(P)
    - C is the ciphertext.
    - P is the plaintext.
    - E is the encryption algorithm.
    - k is the shared symmetric key.

- **Decryption Process:** $P=D_k(C)$
    - D is the decryption algorithm.

Real-World Examples:

1. **File Encryption:** Using a program like 7-Zip or WinRAR to password-protect a file. The password acts as the symmetric key, and anyone with the correct password can decrypt the file.
2. **VPN (Virtual Private Network):** When you connect to a VPN, a secure tunnel is created using symmetric encryption to protect all your internet traffic. The client and the VPN server share a secret key to encrypt and decrypt the data packets.
3. **TLS/SSL Protocol (for bulk data):** While the initial handshake uses asymmetric cryptography, the actual data transfer in a secure web connection (e.g., HTTPS) is encrypted using a session key generated via symmetric cryptography for performance reasons.

Applications:

- **Securing communication channels:** Encrypting data for storage or transmission, like in virtual private networks (VPNs).
- **Bulk data encryption:** Used in file systems, database encryption, and secure backups where large amounts of data need to be encrypted efficiently.
- **Payment systems:** Symmetric encryption is used to secure transactions and customer data in various payment processing systems.

Important Notes / Key Takeaways:

- Uses a single, shared key for both encryption and decryption.
- Faster and more efficient than asymmetric encryption.
- Main challenge is the secure distribution of the key.
- Commonly used algorithms include AES and DES.
- Ideal for bulk data encryption.

---

2. Asymmetric Key Cryptography

Definition: Asymmetric key cryptography, also known as public-key cryptography, is a system that uses a pair of mathematically related keys: a public key and a private key. The

public key can be shared widely, while the private key must be kept secret. Data encrypted with the public key can only be decrypted with the corresponding private key, and vice versa.

Detailed Explanation:

- **Key Pair:** The system is based on a key pair, where one key is used for encryption and the other for decryption. A public key can be freely distributed to anyone you want to communicate with.
- **Public Key:** This key is used by others to encrypt messages intended for you. It can be shared openly without compromising security.
- **Private Key:** This key is known only to the owner and must be kept secret. It is used to decrypt messages that were encrypted with the corresponding public key.
- **Mathematical Trapdoor Function:** The security of asymmetric cryptography is based on a mathematical problem that is easy to solve in one direction (e.g., multiplying two large prime numbers) but extremely difficult to reverse (e.g., factoring the product back into the original primes) without the private key.

Mathematical / Technical Aspect:

- **Encryption Process:** C=E_public_key(P)
- **Decryption Process:** P=D_private_key(C)
- **Digital Signature:** Asymmetric cryptography is also used to create digital signatures. The sender uses their private key to "sign" a message, and the receiver uses the sender's public key to verify the signature. This provides authentication and non-repudiation.

Real-World Examples:

1. **Secure Email:** Services like PGP (Pretty Good Privacy) use public-key cryptography to encrypt email messages. You use the recipient's public key to encrypt the email, and they use their private key to decrypt it.
2. **HTTPS (Secure Web Browsing):** When you connect to a website with HTTPS, the server sends its public key to your browser. Your browser uses this public key to establish a secure connection, and the server uses its private key to decrypt your requests and encrypt its responses.

3. **Cryptocurrencies:** Bitcoin and other cryptocurrencies use asymmetric key pairs. Your public key serves as your wallet address, and your private key is used to authorize transactions, proving that you are the owner of the funds.

Applications:

- **Secure key exchange:** Used to securely distribute a symmetric key for a communication session.
- **Digital signatures:** Verifying the authenticity and integrity of a message or document.
- **Public key infrastructure (PKI):** The foundation of a system that manages and verifies digital certificates and public keys.

Important Notes / Key Takeaways:

- Uses a pair of keys: a public key and a private key.
- Public key for encryption, private key for decryption.
- Slower and computationally more intensive than symmetric encryption.
- Solves the key distribution problem of symmetric cryptography.
- Primarily used for secure key exchange and digital signatures.

---

3. Elliptic Curve Cryptography (ECC)

Definition: Elliptic Curve Cryptography (ECC) is an approach to public-key cryptography based on the algebraic structure of elliptic curves over finite fields. It provides a more efficient and secure alternative to traditional public-key algorithms like RSA, offering similar levels of security with smaller key sizes.

Detailed Explanation:

- **Elliptic Curve over a Finite Field:** Instead of relying on factoring large prime numbers (like RSA), ECC uses the properties of elliptic curves. An elliptic curve is a specific type of curve described by a mathematical equation, and "finite field" means that the coordinates of points on the curve are restricted to a finite set of numbers.
- **The "Elliptic Curve Discrete Logarithm Problem" (ECDLP):** The security of ECC is based on the difficulty of this problem. It is easy to multiply a point on the curve by

a number to get a new point, but it is extremely difficult to figure out what that number was, given the original and final points. This "one-way" function is the basis for the key pair.
- **Smaller Key Sizes:** ECC keys are significantly smaller than RSA keys for the same level of security. For example, a 256-bit ECC key offers comparable security to a 3072-bit RSA key. This leads to reduced computational load and faster performance.

Mathematical / Technical Aspect:

- **General Elliptic Curve Equation:** $y^2=x^3+ax+b$
- **Public Key:** $Q=kG$
  - Q is the public key point.
  - k is the private key (a randomly chosen integer).
  - G is the base point on the curve.
- **The core operation is point multiplication.** Given points P and Q on the curve, we can compute P+Q. The private key is a scalar, which is used to multiply the base point to derive the public key.

Real-World Examples:

1. **Cryptocurrencies:** Many cryptocurrencies, including Bitcoin and Ethereum, use ECC (specifically the secp256k1 curve) to generate public and private key pairs for wallet addresses and to sign transactions.
2. **TLS/SSL:** Modern web browsers and servers use ECC for the TLS/SSL handshake to establish a secure connection. This is often faster and less resource-intensive than using RSA.
3. **Digital Certificates:** Digital certificates issued by Certificate Authorities (CAs) often use ECC keys. These certificates are used to verify the identity of websites and other entities on the internet.

Applications:

- **Mobile and IoT Devices:** Due to their smaller key sizes and lower computational requirements, ECC is ideal for resource-constrained devices like smartphones, smart cards, and IoT sensors.
- **Secure messaging apps:** End-to-end encrypted messaging services like Signal and WhatsApp use ECC to secure communication.

- **Digital signatures and key exchange protocols:** ECC is widely used in various protocols for key exchange and authentication.

Important Notes / Key Takeaways:

- A type of public-key cryptography.
- Based on the mathematics of elliptic curves.
- Offers equivalent security to RSA with much smaller key sizes.
- More efficient and faster, especially on resource-constrained devices.
- Widely used in modern cryptographic protocols and cryptocurrencies.

---

4. Cryptographic Hash Functions: SHA-256

Definition: A cryptographic hash function is a mathematical algorithm that takes an input (or 'message') and converts it into a fixed-size string of bytes. This output, known as a 'hash' or 'digest', has unique properties that make it useful for security applications. SHA-256 is a specific type of cryptographic hash function from the SHA-2 family.

Detailed Explanation:

- **SHA-256 is a one-way function.** It is computationally easy to generate a hash from a given input, but it is practically impossible to reverse the process—to reconstruct the original input from the hash value.
- **The output is a fixed size.** Regardless of the size of the input data (a single word or an entire movie), the SHA-256 hash will always be a 256-bit (32-byte) string.
- **Deterministic:** The same input will always produce the exact same hash output.
- **Avalanche Effect:** Even a tiny change in the input data (e.g., changing one letter or a single pixel in an image) will result in a completely different and unpredictable hash output. This makes it impossible to tamper with the data without changing the hash.

Mathematical / Technical Aspect:

- **Input:** Data of arbitrary length.
- **Hash Function:** SHA-256 algorithm.
- **Output:** A 256-bit (32-byte) hexadecimal string.

- **Example:** SHA256("Hello, World!") results in the hash: d2b5133602d338f0d86926f21c7e1082570072b226e6d78a84618a803738b813
- The function takes the input, processes it through a series of logical and bitwise operations, and produces a final 256-bit hash.

Real-World Examples:

1. **Password Storage:** Websites never store your actual password. Instead, they store a cryptographic hash of your password. When you log in, they hash the password you enter and compare the result with the stored hash. This prevents your password from being exposed if the database is breached.
2. **File Integrity Check:** You can download a large file and compare its SHA-256 hash with a hash provided by the website. If the hashes match, you can be sure the file was not corrupted or tampered with during the download.
3. **Blockchain Technology:** SHA-256 is the core hash function used in Bitcoin. Blocks in the blockchain are linked together by including the hash of the previous block, creating an immutable and tamper-proof ledger.

Applications:

- **Data integrity verification:** Ensuring that a file or message has not been altered.
- **Password security:** Storing and verifying passwords securely.
- **Digital signatures:** A hash of the document is signed rather than the entire document, which is much more efficient.
- **Blockchain and cryptocurrency:** Creating an unchangeable record of transactions.

Important Notes / Key Takeaways:

- Transforms data of any size into a fixed-size hash.
- SHA-256 is one-way (irreversible).
- Even a small change in input drastically changes the output (avalanche effect).
- Used for data integrity, password security, and in blockchain.

---

5. Digital Signature Algorithm (DSA)

Definition: The Digital Signature Algorithm (DSA) is a U.S. government standard for digital signatures. It is a cryptographic method used to verify the authenticity and

integrity of a digital message or document. A digital signature is not the same as a physical signature; it is a mathematical scheme for proving the authenticity of digital messages or documents.

Detailed Explanation:

- **It uses asymmetric cryptography.** The process involves two keys: a private key for signing and a public key for verifying the signature. The sender uses their private key to create a signature, and the receiver uses the sender's public key to verify that the signature is valid.
- **Signing process:** The sender takes the document, computes its cryptographic hash, and then encrypts the hash with their private key. The resulting encrypted hash is the digital signature.
- **Verification process:** The receiver computes the hash of the received document. They then use the sender's public key to decrypt the signature. If the two hash values match, it proves that the document has not been altered and that it was indeed signed by the owner of the private key.
- **Provides authenticity, integrity, and non-repudiation.**
    - **Authenticity:** The receiver can be sure the message came from the claimed sender.
    - **Integrity:** The receiver can be sure the message was not altered after it was signed.
    - **Non-repudiation:** The sender cannot later deny having signed the message.

Mathematical / Technical Aspect:

- **Key generation:** Involves selecting a prime number p, a generator g, and a private key x. The public key y is calculated as y=gxpmodp.
- **Signing:** The signature consists of a pair of numbers, (r, s), calculated using the private key x and a random number k.
- **Verification:** The receiver computes two values, v1 and v2, using the sender's public key y, and checks if v1 = v2 to confirm the signature's validity.

Real-World Examples:

1. **Software Downloads:** When you download a software installer, the developer often provides a digital signature. You can use this to verify that the software is from the legitimate developer and has not been tampered with by a third party.

2. **Email Encryption:** Digital signatures are used in secure email systems to authenticate the sender and ensure the message's integrity.
3. **Secure Software Updates:** Operating systems and applications use digital signatures to verify that software updates are from a trusted source and have not been maliciously modified.

Applications:

- **Electronic document signing:** Legally binding documents can be signed digitally.
- **Secure software distribution:** Verifying the authenticity of software from a trusted vendor.
- **Authentication:** Proving the identity of a sender or a device in a network.

Important Notes / Key Takeaways:

- A standard for creating and verifying digital signatures.
- Uses asymmetric cryptography (private key for signing, public key for verifying).
- Ensures authenticity, integrity, and non-repudiation.
- Does not encrypt the entire message, only its hash.
- Used in software distribution, secure email, and electronic transactions.

---

6. Merkle Trees

Definition: A Merkle tree, also known as a hash tree, is a data structure used in computer science and cryptography for verifying data integrity and consistency. It is a tree-like structure where every leaf node is a hash of a data block, and every non-leaf node is a hash of its child nodes' hashes. This hierarchical hashing allows for efficient and secure verification of large data sets.

Detailed Explanation:

- **Data Integrity:** Merkle trees are fundamentally about verifying data. They allow a party to quickly and efficiently verify that a specific piece of data is part of a larger set without needing to download the entire set.
- **Root Hash:** The hash at the top of the tree is called the Merkle root or root hash. This single hash uniquely represents the entire set of underlying data. If even one

bit of data in any of the leaf nodes is changed, it will change the hash of that leaf, which will cascade up the tree, and ultimately change the root hash.

- **Proof of Inclusion:** To prove that a data block exists within the set, you only need to provide the hashes along the "path" from that data block up to the Merkle root. This proof is much smaller than the total data set.
- **Efficiency:** Because you only need to compare hashes, Merkle trees significantly reduce the amount of data that needs to be transmitted for verification, which is crucial for systems with limited bandwidth or large data sets.

Mathematical / Technical Aspect:

- The core operation is a series of concatenations and hash computations.
- H(X) denotes the hash of X.
- The hash of a non-leaf node is the hash of the concatenation of its children's hashes: H(Node)=H(H(Child1)+H(Child2)).
- **Example:**
  - `H_0-0=H(textDataBlock0)`
  - `H_0-1=H(textDataBlock1)`
  - `H_0=H(H_0-0+H_0-1)`
  - The process continues up to the root.

Real-World Examples:

1. **Blockchain Technology:** Bitcoin and other blockchains use Merkle trees to summarize all the transactions in a block. The Merkle root is stored in the block header. This allows for fast verification of whether a transaction is included in a block without downloading all of the block's transactions.
2. **Peer-to-Peer Networks:** Systems like BitTorrent use Merkle trees. When you download a file, you get a Merkle root from a trusted source. You can then download different parts of the file from multiple peers and verify that each part is correct using the Merkle tree structure.
3. **Git (Version Control System):** Git uses a similar concept (Merkle DAGs - Directed Acyclic Graphs) to track changes and versions of files. Each commit is a hash of its content, and the history is a chain of hashes, providing a secure and verifiable record.

Applications:

- **Cryptocurrencies and blockchain:** Verifying transactions and ensuring the integrity of the ledger.
- **Distributed systems:** Ensuring data consistency across multiple nodes.
- **Peer-to-peer file sharing:** Verifying the integrity of file parts downloaded from different sources.
- **Secure data synchronization:** Efficiently checking for changes between two data sets.

Important Notes / Key Takeaways:

- A tree-like data structure made of hashes.
- Efficiently verifies data integrity in large data sets.
- The root hash uniquely represents all the data.
- Used extensively in blockchain to verify transactions.
- Allows for "light client" verification without full data download.

Summary Table

Main Topic Name	Brief Definition	3 Real-World Example
Symmetric Key Cryptography	An encryption system where the same secret key is used by both the sender and the receiver for encryption and decryption. The key must be securely shared beforehand. It is computationally fast and efficient, making it ideal for encrypting large volumes of data.	1. Password-protecting a ZIP file. 2. Securing a VPN connection. 3. Encrypting data on a hard drive using a password.
Asymmetric Key Cryptography	A cryptographic system that uses a pair of mathematically linked keys: a public key for encryption and a private key for decryption. The public key can be freely shared, while the private key must remain secret. It solves the key distribution problem but is computationally slower.	1. Secure email with PGP. 2. The HTTPS protocol handshake on websites. 3. Signing cryptocurrency transactions with a private key.

Elliptic Curve Cryptography (ECC)	A type of public-key cryptography that uses the algebraic structure of elliptic curves over finite fields. It provides a high level of security with significantly smaller key sizes compared to other public-key algorithms, making it more efficient and suitable for modern, resource-constrained devices.	1. Public and private keys for Bitcoin wallets. 2. Modern TLS/SSL protocols for web security. 3. Secure messaging apps like Signal and WhatsApp.
Cryptographic Hash Functions: SHA-256	An algorithm that takes an input of any size and produces a fixed-size, irreversible string of characters (the hash). It is deterministic, meaning the same input always yields the same hash, and is highly sensitive to input changes, making it ideal for verifying data integrity.	1. Storing password hashes on a website. 2. Verifying the integrity of a downloaded software file. 3. Hashing transactions in a blockchain.
Digital Signature Algorithm (DSA)	A standard for creating and verifying digital signatures, which are cryptographic schemes used to verify the authenticity and integrity of a digital message. It uses a sender's private key to sign a hash of the message, which can then be verified by anyone using the sender's public key.	1. Signing software updates to prove their authenticity. 2. Authenticating senders in secure email systems. 3. Legally binding electronic signatures on documents.
Merkle Trees	A tree-like data structure where each leaf node contains the hash of a data block, and each non-leaf node contains the hash of its children. This structure allows for efficient and secure verification of the integrity of large data sets by simply checking a single root hash.	1. Verifying transactions in Bitcoin blocks. 2. Checking the integrity of files in BitTorrent. 3. Used in Git to track file changes securely.

Thank-You

We sincerely appreciate your time and attention in reviewing this document. Your feedback and support mean a lot to us, and we look forward to building something valuable together.

PUNE UNIVERSITY

Visit site to access further Notes!!!

✨ **Why Choose Us?**

- 📚 **Free & Updated Notes for All Subjects**
- 🎯 **Exam-Focused Content**
- 🔍 **Easy Navigation by Unit & Topic**
- 💻 **Works on Mobile & Desktop**

Branches: **CS | IT | AI&DS | AI&ML**

Notes By: puneuniversity.netlify.app

🔗 **Visit us: https://cutt.ly/IrHFgvYI**